

TRƯỜNG ĐẠI HỌC KHOA HỌC
KHOA CÔNG NGHỆ THÔNG TIN



ĐỀ TÀI TIỂU LUẬN
ĐIỀU KHIỂN THIẾT BỊ GIA DỤNG
QUA WI-FI VỚI ESP32

Giảng viên hướng dẫn Võ Việt Dũng

Định hướng triển khai Điều khiển thiết bị gia dụng qua Wi-Fi với
ESP32

Nền tảng điều khiển Blynk Web/Mobile và Telegram

HUẾ, THÁNG 4 NĂM 2026

MỤC LỤC

PHẦN MỞ ĐẦU	1
1. Tính cấp thiết của đề tài	1
2. Mục tiêu và nhiệm vụ nghiên cứu	1
3. Đối tượng và phạm vi nghiên cứu	2
PHẦN NỘI DUNG	3
1. Tổng quan về ESP32	4
2. Công nghệ sử dụng trong hệ thống	6
3. Thiết kế và triển khai hệ thống	9
4. Xây dựng chương trình điều khiển	12
5. Kết quả mô phỏng và đánh giá	14
KẾT LUẬN	20
TÀI LIỆU THAM KHẢO	21

PHẦN MỞ ĐẦU

1. TÍNH CẤP THIẾT CỦA ĐỀ TÀI

Cùng với tiến trình chuyển đổi số, Internet of Things (IoT) không còn dừng lại ở mức khái niệm mà đã trở thành nền tảng quan trọng cho nhiều hệ thống giám sát và điều khiển thông minh. Điểm nổi bật của IoT là khả năng kết nối các đối tượng vật lý với hạ tầng mạng, cho phép thu thập dữ liệu, đưa ra quyết định và điều khiển thiết bị theo thời gian thực từ nhiều môi trường khác nhau.

Trong không gian sống hiện đại, nhu cầu quản lý các thiết bị điện gia dụng theo hướng thuận tiện, tiết kiệm và an toàn ngày càng rõ rệt. Những thao tác quen thuộc như bật đèn trước khi về nhà, tắt quạt khi quên không sử dụng, hay theo dõi trạng thái thiết bị bằng điện thoại đều là những tình huống thể hiện rõ lợi ích của nhà thông minh ở quy mô nhỏ. Đây là bài toán phù hợp để sinh viên tiếp cận, vì vừa gần gũi với thực tế vừa có thể triển khai trên mô hình thí nghiệm với chi phí hợp lý.

ESP32 là nền tảng đáp ứng tốt yêu cầu đó nhờ được tích hợp sẵn Wi-Fi, có khả năng xử lý ổn định, hỗ trợ nhiều chân GPIO và tương thích với nhiều thư viện phát triển IoT phổ biến. Khi kết hợp ESP32 với Blynk và Telegram Bot, hệ thống không chỉ có giao diện điều khiển trực quan trên web hoặc điện thoại mà còn có thêm một kênh tương tác linh hoạt thông qua tin nhắn, giúp mở rộng khả năng sử dụng trong điều kiện mạng và thiết bị khác nhau.

Xuất phát từ các lý do trên, đề tài “Điều khiển thiết bị gia dụng qua Wi-Fi sử dụng ESP32” được lựa chọn nhằm xây dựng một mô hình điều khiển đèn, quạt từ xa, có khả năng giám sát môi trường và đồng bộ trạng thái giữa nhiều nền tảng. Đề tài vừa có giá trị học thuật trong việc củng cố kiến thức về IoT, vừa có ý nghĩa ứng dụng khi phản ánh đúng cách một hệ thống Smart Home cơ bản được hình thành và vận hành.

2. MỤC TIÊU VÀ NHIỆM VỤ NGHIÊN CỨU

Đề tài hướng đến việc thiết kế một hệ thống điều khiển thiết bị gia dụng cơ bản dựa trên kết nối Wi-Fi, trong đó ESP32 giữ vai trò bộ xử lý trung tâm, tiếp nhận lệnh từ người dùng, điều khiển thiết bị đầu ra và cập nhật trạng thái lên các nền tảng giám

sát. Hệ thống cần đáp ứng tính dễ sử dụng, khả năng mở rộng và phù hợp với điều kiện học tập, thử nghiệm trong môi trường mô phỏng.

Mục tiêu cụ thể:

- Xây dựng được mô hình điều khiển bật/tắt đèn và quạt từ xa thông qua kết nối Wi-Fi.
- Tạo giao diện điều khiển trực quan trên Blynk Web/Mobile và bổ sung kênh điều khiển qua Telegram Bot.
- Giám sát một số thông số môi trường cơ bản như nhiệt độ và cường độ sáng để hỗ trợ tự động hóa ở mức đơn giản.
- Đảm bảo trạng thái thiết bị được cập nhật nhất quán khi người dùng thao tác từ nhiều nền tảng khác nhau.

Nhiệm vụ nghiên cứu:

- Khảo sát cơ sở lý thuyết liên quan đến IoT, ESP32, Blynk, Telegram Bot API, PlatformIO, Wokwi và giao thức HTTP.
- Thiết kế sơ đồ kết nối giữa ESP32, cảm biến DHT22, cảm biến ánh sáng LDR, LED mô phỏng đèn và LED mô phỏng quạt.
- Xây dựng chương trình điều khiển để thiết bị nhận lệnh từ Blynk và Telegram, đồng thời phản hồi trạng thái hệ thống.
- Mô phỏng toàn bộ hệ thống trên Wokwi nhằm kiểm tra tính đúng đắn của kết nối phần cứng và logic xử lý phần mềm.
- Đánh giá kết quả mô phỏng, chỉ ra ưu điểm, hạn chế và đề xuất hướng phát triển cho phiên bản triển khai thực tế.

3. ĐỐI TƯỢNG VÀ PHẠM VI NGHIÊN CỨU

Đối tượng nghiên cứu của đề tài bao gồm vi điều khiển ESP32, các thiết bị đầu ra mô phỏng cho đèn và quạt, các cảm biến môi trường cơ bản như DHT22 và LDR, cùng những nền tảng hỗ trợ điều khiển từ xa như Blynk và Telegram. Trên phương diện kỹ thuật, đề tài tập trung vào cách thiết kế một hệ thống IoT quy mô nhỏ có thể vận hành ổn định trên mạng Wi-Fi và đáp ứng được các thao tác điều khiển phổ biến của người dùng.

Phạm vi triển khai của đề tài được giới hạn ở mức mô hình Smart Home cơ bản. Hệ thống chủ yếu được kiểm thử trong môi trường mô phỏng Wokwi, chưa đi sâu vào bài toán điều khiển tải công suất lớn bằng relay công nghiệp, bảo mật nhiều lớp, lưu trữ dữ liệu dài hạn hay tích hợp nhiều lớp dịch vụ đám mây. Ngoài ra, đề tài cũng không mở rộng sang các giao thức có tính publish-subscribe như MQTT, vì mục tiêu trọng tâm là làm rõ luồng điều khiển và giám sát qua Wi-Fi ở mức nền tảng.

- Các thiết bị nghiên cứu chính: ESP32, DHT22, LDR, LED mô phỏng đèn, LED mô phỏng quạt và nút nhấn thủ công.
- Phần điều khiển tập trung vào thao tác bật/tắt thiết bị, hiển thị trạng thái và đưa ra phản hồi khi môi trường thay đổi.
- Môi trường thử nghiệm chính là Wokwi kết hợp PlatformIO, phù hợp cho việc học tập, kiểm tra logic và hoàn thiện tài liệu học thuật.

PHẦN NỘI DUNG

1. TỔNG QUAN VỀ ESP32

1.1. Khái quát về nền tảng ESP32

ESP32 là dòng vi điều khiển do Espressif Systems phát triển, được sử dụng rộng rãi trong các dự án nhúng và Internet of Things nhờ tích hợp sẵn Wi-Fi và Bluetooth ngay trên cùng một chip. So với nhiều giải pháp vi điều khiển phổ thông, ESP32 giảm đáng kể số lượng linh kiện bổ sung cần thiết cho bài toán kết nối mạng, từ đó giúp mô hình đơn giản hơn và tiết kiệm chi phí triển khai.

Với định hướng phục vụ các ứng dụng kết nối thời gian thực, ESP32 được thiết kế để xử lý đồng thời nhiều tác vụ như duy trì kết nối Wi-Fi, nhận lệnh điều khiển, đọc dữ liệu cảm biến và cập nhật trạng thái thiết bị. Chính sự cân bằng giữa hiệu năng, khả năng mở rộng và giá thành là lý do khiến ESP32 trở thành nền tảng phù hợp cho các bài tập lớn, đồ án hoặc mô hình Smart Home ở quy mô nhỏ và vừa.

1.2. Cấu trúc phần cứng và hiệu năng xử lý

ESP32 thường được nhắc đến với bộ xử lý lõi kép Xtensa LX6 có xung nhịp lên đến 240 MHz, bộ nhớ SRAM 520 KB và bộ nhớ flash từ 4 MB trở lên tùy phiên bản. Cấu hình này đủ để đáp ứng các bài toán điều khiển thiết bị gia dụng thông minh, nơi bộ điều khiển cần chạy song song nhiều công việc như giao tiếp mạng, xử lý lệnh người dùng và cập nhật dữ liệu cảm biến theo chu kỳ.

Ngoài năng lực tính toán, ESP32 còn hỗ trợ nhiều chuẩn giao tiếp phần cứng như UART, SPI, I2C, PWM, ADC và DAC. Điều này đặc biệt quan trọng trong các hệ thống IoT vì nó cho phép nhà phát triển kết nối linh hoạt với cảm biến nhiệt độ, cảm biến ánh sáng, màn hình hiển thị, module relay hoặc nhiều linh kiện ngoại vi khác mà không phải thay đổi nền tảng trung tâm.

1.3. Khả năng kết nối không dây và tiết kiệm năng lượng

Điểm mạnh nổi bật nhất của ESP32 nằm ở khả năng kết nối Wi-Fi 2.4 GHz chuẩn 802.11 b/g/n, đồng thời hỗ trợ Bluetooth Classic và BLE. Nhờ đó, một thiết bị ESP32 có thể đảm nhận vai trò kết nối với router gia đình để truy cập internet, hoặc giao tiếp gần với điện thoại và các thiết bị ngoại vi khác khi bài toán yêu cầu.

Bên cạnh hiệu năng mạng, ESP32 còn cung cấp nhiều chế độ tiết kiệm năng lượng như modem sleep, light sleep và deep sleep. Dù đề tài hiện tại sử dụng nguồn cấp ổn định trong môi trường mô phỏng, việc hiểu và nắm được các chế độ năng lượng vẫn là tiền đề quan trọng nếu hệ thống được mở rộng sang các ứng dụng dùng pin hoặc yêu cầu hoạt động liên tục trong thời gian dài.

1.4. Vai trò của ESP32 trong mô hình điều khiển thiết bị

Trong đề tài này, ESP32 giữ vị trí trung tâm của toàn bộ hệ thống. Vi điều khiển có nhiệm vụ nhận lệnh từ Blynk hoặc Telegram, phân tích nội dung điều khiển, kích hoạt chân GPIO tương ứng để bật hoặc tắt thiết bị, đồng thời gửi lại trạng thái hoạt động để giao diện điều khiển hiển thị chính xác.

Ngoài chức năng điều khiển, ESP32 còn đọc dữ liệu từ DHT22 và LDR nhằm hỗ trợ giám sát môi trường và đưa ra hành vi tự động ở mức cơ bản. Nói cách khác, ESP32 vừa là điểm hội tụ dữ liệu, vừa là bộ ra quyết định của hệ thống, giúp kết nối hiệu quả giữa người dùng, cảm biến và thiết bị đầu ra.

2. CÔNG NGHỆ SỬ DỤNG TRONG HỆ THỐNG

2.1. PlatformIO trên Visual Studio Code

PlatformIO được lựa chọn làm môi trường phát triển vì cung cấp cấu trúc dự án rõ ràng, quản lý thư viện thuận tiện và hỗ trợ tốt cho ESP32. So với cách viết chương trình trong các môi trường đơn giản, PlatformIO giúp phân tách tệp cấu hình, mã nguồn và phụ thuộc thư viện một cách chuyên nghiệp hơn, nhờ đó việc bảo trì dự án trở nên dễ dàng khi hệ thống mở rộng thêm nhiều thành phần.

Khi tích hợp với Visual Studio Code, PlatformIO còn mang lại lợi ích ở khía cạnh soạn thảo mã nguồn, gợi ý cú pháp, kiểm tra lỗi và biên dịch nhanh. Đây là lựa chọn phù hợp cho các dự án học thuật cần đồng thời viết mã, quản lý cấu hình và lưu trữ phiên bản tài liệu kỹ thuật.

2.2. Wokwi

Wokwi là nền tảng mô phỏng trực tuyến hỗ trợ nhiều vi điều khiển như Arduino, ESP32 và Raspberry Pi Pico. Đối với đề tài này, Wokwi giúp dựng sơ đồ mạch, mô phỏng cảm biến, quan sát trạng thái các chân vào/ra và kiểm thử chương trình trước khi triển khai trên phần cứng thật.

Lợi ích lớn của Wokwi nằm ở khả năng rút ngắn thời gian thử nghiệm. Người thực hiện có thể thay đổi chân kết nối, chỉnh tham số mô phỏng hoặc kiểm tra luồng xử lý mà không cần lắp ráp phần cứng nhiều lần. Điều này đặc biệt hữu ích trong giai đoạn hoàn thiện tiểu luận, khi tính chính xác của sơ đồ mạch và logic chương trình cần được xác minh rõ ràng.

2.3. Blynk Web/Mobile

Blynk là nền tảng IoT cho phép xây dựng dashboard điều khiển trên web và ứng dụng di động. Trong mô hình đề tài, Blynk đóng vai trò giao diện điều khiển chính, nơi người dùng thực hiện thao tác bật hoặc tắt đèn, quạt, đồng thời theo dõi giá trị nhiệt độ, ánh sáng và nội dung trạng thái hệ thống.

Thế mạnh của Blynk là cơ chế Virtual Pins, nhờ đó dữ liệu điều khiển và dữ liệu hiển thị được tách khỏi chân phần cứng thật của ESP32. Cách tiếp cận này giúp hệ

thông dễ mở rộng, dễ đồng bộ và trực quan hơn khi thiết kế giao diện cho người dùng cuối.

2.4. Telegram Bot

Telegram Bot được sử dụng như một kênh điều khiển phụ nhưng có tính thực tế cao. Thông qua bot, người dùng chỉ cần gửi các lệnh dạng văn bản như bật đèn, tắt đèn, bật quạt, tắt quạt hoặc yêu cầu báo trạng thái để điều khiển hệ thống từ xa mà không cần mở giao diện dashboard.

Ưu điểm của kênh Telegram là thao tác nhanh, phù hợp khi người dùng chỉ cần điều khiển tức thời hoặc nhận thông báo phản hồi. Trong phạm vi đề tài, Telegram cũng là ví dụ điển hình cho việc tích hợp một dịch vụ bên thứ ba vào hệ thống IoT mà vẫn duy trì được luồng xử lý đơn giản và dễ hiểu.

2.5. Giao thức HTTP trong hệ thống

Về mặt nguyên lý, các thành phần trong mô hình tương tác theo cơ chế yêu cầu - phản hồi. Ở đó, Blynk Cloud và Telegram Bot API tiếp nhận hoặc phát sinh yêu cầu từ người dùng, còn ESP32 đọc dữ liệu điều khiển, xử lý và gửi phản hồi trạng thái trở lại. Dù người dùng có thể nhìn hệ thống như một mô hình điều khiển trực tiếp, ở tầng triển khai đây vẫn là quá trình trao đổi dữ liệu qua các phiên làm việc mạng dựa trên HTTP hoặc HTTPS.

HTTP được lựa chọn vì dễ triển khai, phù hợp với quy mô bài toán và thuận lợi cho việc học tập. So với các giao thức thiên về publish-subscribe như MQTT, HTTP giúp người học quan sát rõ hơn từng bước: gửi lệnh, nhận lệnh, xử lý, phản hồi và đồng bộ giao diện. Điều này phù hợp với mục tiêu của đề tài là trình bày một mô hình dễ triển khai nhưng vẫn phản ánh đúng tinh thần của hệ thống IoT hiện đại.

2.6. Yêu cầu đồng bộ trạng thái

Khi cùng một thiết bị có thể được điều khiển từ Blynk, Telegram và nút nhấn vật lý, một yêu cầu quan trọng phát sinh là phải đồng bộ trạng thái giữa các kênh. Nếu đèn được bật từ Telegram nhưng giao diện Blynk không cập nhật, người dùng dễ đưa ra thao tác lặp hoặc nhận định sai về tình trạng thực tế của hệ thống.

Bởi vậy, mỗi thao tác điều khiển trong chương trình đều cần được xem như một sự kiện trung tâm: trạng thái thiết bị phải được ghi nhận tại ESP32, sau đó cập nhật ngược lên Blynk và phản hồi về Telegram. Tư duy đồng bộ này không chỉ giúp hệ thống hoạt động nhất quán mà còn tạo nền tảng cho việc mở rộng sau này khi số lượng thiết bị tăng lên.

Bảng 1. Vai trò của các thành phần chính trong hệ thống

Thành phần	Vai trò trong hệ thống	Ghi chú triển khai
ESP32	Bộ điều khiển trung tâm	Kết nối Wi-Fi, nhận lệnh và điều khiển GPIO
Blynk	Giao diện điều khiển chính	Dùng trên web và điện thoại, hiển thị trạng thái
Telegram Bot	Kênh điều khiển bổ sung	Nhận lệnh bằng tin nhắn và gửi phản hồi tức thời
Wokwi	Môi trường mô phỏng	Kiểm tra sơ đồ mạch và luồng xử lý phần mềm
DHT22	Cảm biến nhiệt độ/độ ẩm	Cung cấp dữ liệu phục vụ giám sát và điều khiển quạt
LDR	Cảm biến ánh sáng	Đánh giá độ sáng môi trường để hỗ trợ bật đèn

3. THIẾT KẾ MÔ HÌNH VÀ KIẾN TRÚC HỆ THỐNG

3.1. Mô tả bài toán điều khiển

Bài toán của hệ thống là cho phép người dùng bật hoặc tắt đèn, quạt từ xa thông qua mạng Wi-Fi, đồng thời quan sát được một số thông số môi trường cơ bản để hỗ trợ giám sát. Trong điều kiện ánh sáng giảm xuống dưới ngưỡng thiết lập, đèn có thể được kích hoạt tự động; tương tự, quạt có thể được bật khi nhiệt độ vượt quá giá trị cho phép. Cơ chế này biến hệ thống từ một mô hình điều khiển thuần túy trở thành mô hình điều khiển có yếu tố tự động hóa.

3.2. Thành phần phần cứng của mô hình

- ESP32 DevKit được dùng làm bộ điều khiển trung tâm, đảm nhiệm nhiệm vụ kết nối mạng, đọc cảm biến và điều khiển thiết bị.
- Cảm biến DHT22 được sử dụng để đo nhiệt độ và độ ẩm môi trường, cung cấp dữ liệu đầu vào phục vụ hiển thị và kích hoạt quạt tự động.
- Cảm biến ánh sáng LDR cho phép đánh giá điều kiện chiếu sáng trong không gian mô phỏng, từ đó hỗ trợ logic bật đèn khi môi trường tối.
- LED đỏ đại diện cho đèn chiếu sáng; LED xanh đóng vai trò mô phỏng quạt trong hệ thống. Hai nút nhấn được bổ sung để mô phỏng điều khiển thủ công trực tiếp từ phần cứng.

Thiết bị	GPIO đề xuất	Chức năng
DHT22	GPIO15	Đọc nhiệt độ và độ ẩm
LDR	GPIO34	Đọc cường độ ánh sáng môi trường
LED đỏ	GPIO22	Mô phỏng đèn chiếu sáng
LED xanh	GPIO23	Mô phỏng quạt
Nút nhấn 1	GPIO19	Bật/tắt đèn thủ công
Nút nhấn 2	GPIO18	Bật/tắt quạt thủ công

Bảng 2. Ánh xạ kết nối phần cứng đề xuất trong môi trường mô phỏng

3.3. Kiến trúc tổng thể của hệ thống

Ở góc độ kiến trúc, mô hình có thể được chia thành ba lớp: lớp giao tiếp người dùng, lớp xử lý trung tâm và lớp thiết bị/cảm biến. Lớp giao tiếp người dùng gồm Blynk và Telegram, nơi phát sinh yêu cầu điều khiển. Lớp xử lý trung tâm là ESP32, nơi tiếp nhận và ra quyết định. Lớp cuối cùng là các thiết bị đầu ra và cảm biến chịu trách nhiệm phản ánh kết quả điều khiển cũng như cung cấp thông tin về môi trường.

Cách phân lớp này giúp hệ thống rõ ràng về chức năng và dễ mở rộng. Nếu sau này cần bổ sung thêm máy bơm, cảm biến khí gas hoặc cảm biến chuyển động, người thực hiện chỉ cần mở rộng lớp thiết bị và cập nhật logic xử lý tại ESP32 mà không phải thay đổi toàn bộ mô hình.

3.4. Luồng xử lý điều khiển và phản hồi

Khi người dùng thao tác trên Blynk, ứng dụng gửi trạng thái điều khiển lên máy chủ Blynk và ESP32 đọc giá trị mới thông qua Virtual Pins. Với Telegram, ESP32 định kỳ kiểm tra tin nhắn mới, phân tích câu lệnh nhận được rồi thực thi hành động tương ứng. Sau khi điều khiển xong, hệ thống phải ngay lập tức đồng bộ lại trạng thái đèn, quạt và gửi phản hồi để người dùng biết rằng lệnh đã được xử lý thành công.

Một điểm quan trọng trong thiết kế là mọi thao tác điều khiển đều hội tụ về ESP32. Nhờ vậy, việc xử lý xung đột trạng thái được đơn giản hóa: ESP32 luôn là nơi ghi nhận trạng thái cuối cùng và chịu trách nhiệm cập nhật lại cho các nền tảng khác.

3.5. Kịch bản tự động hóa cơ bản

Ngoài chế độ điều khiển trực tiếp từ người dùng, hệ thống còn áp dụng một số quy tắc tự động đơn giản để tăng tính ứng dụng. Chẳng hạn, khi nhiệt độ vượt ngưỡng định trước, chương trình có thể kích hoạt quạt để cải thiện thông gió. Khi cường độ sáng xuống thấp, đèn sẽ được bật để mô phỏng phản ứng tự nhiên của một hệ thống nhà thông minh.

Các quy tắc này được giữ ở mức cơ bản để đảm bảo tính minh bạch trong báo cáo. Tuy nhiên, về mặt phát triển, chúng có thể được mở rộng thành các ngưỡng linh hoạt, biểu đồ lịch sử hoặc thậm chí là các luật kết hợp nhiều cảm biến nếu hệ thống được triển khai thực tế.

3.6. Cấu hình phần mềm và nguyên tắc an toàn thông tin

Phần mềm của hệ thống được tổ chức theo hướng tách riêng các tham số cấu hình như tên mạng Wi-Fi, mã xác thực Blynk, token Telegram và các thông số chân kết nối. Việc tách cấu hình giúp mã nguồn dễ bảo trì, đồng thời hạn chế rủi ro khi chia sẻ tài liệu hoặc công bố chương trình trong môi trường học tập.

Trong báo cáo này, các khóa truy cập không được trình bày công khai dưới dạng giá trị thật. Đây là nguyên tắc cần thiết trong quá trình viết tiểu luận kỹ thuật, vì tài liệu thường được chia sẻ giữa nhiều người và có thể được dùng lại ở các lần chỉnh sửa sau.

4. XÂY DỰNG CHƯƠNG TRÌNH ĐIỀU KHIỂN

4.1. Tổ chức mã nguồn và các thư viện chính

Chương trình điều khiển được xây dựng trong môi trường PlatformIO với các thư viện cốt lõi gồm WiFi, BlynkSimpleEsp32, DHT, ArduinoJson và UniversalTelegramBot. Việc sử dụng thư viện giúp rút ngắn thời gian phát triển vì nhiều tác vụ kết nối, xác thực và xử lý dữ liệu đã được đóng gói sẵn theo hướng tiện dụng cho ESP32.

Mã nguồn được tổ chức quanh hai nhóm hàm chính: nhóm khởi tạo trong phần setup và nhóm vận hành lặp lại trong loop. Ngoài ra còn có các hàm chức năng để gửi trạng thái hệ thống, nhận lệnh từ Telegram, xử lý sự kiện Blynk, đọc nút nhấn và kiểm tra điều kiện tự động.

4.2. Khởi tạo kết nối Wi-Fi và dịch vụ nền

Ở bước khởi tạo, ESP32 cần cấu hình các chân vào/ra, kích hoạt cảm biến và thực hiện kết nối Wi-Fi. Khi mạng đã sẵn sàng, chương trình tiếp tục tạo phiên làm việc với Blynk và Telegram, đồng thời chuẩn bị bộ hẹn giờ để gửi dữ liệu theo chu kỳ hoặc kiểm tra lại kết nối nếu có sự cố phát sinh.

Cách khởi tạo theo thứ tự giúp hệ thống tránh lỗi dây chuyền. Nếu Wi-Fi chưa sẵn sàng mà chương trình cố truy cập Blynk hoặc Telegram, khả năng thất bại cao và khó kiểm soát. Vì vậy, kết nối mạng luôn được xem như điều kiện tiên quyết trước khi kích hoạt các dịch vụ điều khiển bên trên.

4.3. Điều khiển qua Blynk

Trong Blynk, mỗi nút điều khiển trên dashboard được ánh xạ với một Virtual Pin. Khi người dùng chuyển trạng thái nút, ESP32 nhận giá trị mới, cập nhật biến trạng thái nội bộ rồi xuất mức tín hiệu tương ứng ra chân điều khiển đèn hoặc quạt. Sau khi điều khiển, trạng thái mới tiếp tục được ghi ngược lên dashboard để giao diện phản ánh chính xác tình trạng hiện thời của hệ thống.

Cách làm này cho phép tách biệt rõ giữa giao diện và phần cứng. Blynk chỉ quản lý hành vi điều khiển ở mức logic, còn quyền quyết định cuối cùng vẫn nằm tại ESP32.

Đây là cách tiếp cận phù hợp cho các hệ thống cần duy trì tính nhất quán khi có nhiều nguồn lệnh cùng tồn tại.

4.4. Điều khiển qua Telegram

Đối với Telegram, chương trình định kỳ truy vấn bot để kiểm tra xem có tin nhắn mới hay không. Nội dung tin nhắn được so khớp với các lệnh đã quy ước trước, ví dụ bật đèn, tắt đèn, bật quạt, tắt quạt hoặc yêu cầu báo trạng thái. Sau khi hoàn thành tác vụ, ESP32 gửi một tin nhắn phản hồi nhằm xác nhận kết quả xử lý cho người dùng.

So với giao diện dạng nút bấm, Telegram phù hợp với kịch bản điều khiển nhanh và nhẹ. Người dùng có thể thao tác ở bất kỳ đâu chỉ với một cuộc trò chuyện đang mở, điều này làm tăng tính linh hoạt của hệ thống mà không cần phát triển thêm một giao diện riêng biệt.

4.5. Xử lý nút nhấn vật lý và đồng bộ trạng thái

Hai nút nhấn vật lý được thêm vào nhằm mô phỏng thao tác cục bộ tại thiết bị. Khi người dùng nhấn nút, trạng thái đèn hoặc quạt sẽ thay đổi ngay trên phần cứng. Tuy nhiên, để hệ thống không bị sai lệch giữa các nền tảng, chương trình phải tiếp tục cập nhật lại lên Blynk và Telegram sau khi nút nhấn được xử lý.

Nội dung này cho thấy vai trò trung tâm của ESP32 trong việc điều phối trạng thái. Dù lệnh đến từ đâu, biến trạng thái của hệ thống vẫn cần được cập nhật thống nhất trước khi phản ánh ra bên ngoài. Nếu bỏ qua bước đồng bộ, một hệ thống có nhiều kênh điều khiển rất dễ rơi vào tình trạng “đèn đang sáng nhưng giao diện báo tắt”.

4.6. Điều khiển tự động theo cảm biến

Bên cạnh điều khiển thủ công, chương trình định kỳ đọc nhiệt độ từ DHT22 và cường độ sáng từ LDR. Khi giá trị cảm biến vượt qua ngưỡng đã thiết lập, hệ thống có thể tự kích hoạt đèn hoặc quạt. Cách tiếp cận này làm tăng mức thông minh cho mô hình mà không đòi hỏi thuật toán phức tạp.

Trong quá trình xây dựng tiểu luận, các ngưỡng điều khiển được giữ ở mức đơn giản để dễ quan sát kết quả mô phỏng. Khi chuyển sang triển khai thực tế, các ngưỡng

này nên được tinh chỉnh theo môi trường sử dụng, đồng thời có thể bổ sung cơ chế trễ hoặc lọc nhiễu để tránh bật/tắt thiết bị liên tục.

4.7. Kiểm tra lỗi và tái kết nối

Một hệ thống IoT muốn hoạt động ổn định cần dự liệu khả năng mất Wi-Fi hoặc gián đoạn kết nối đến dịch vụ đám mây. Vì vậy, chương trình nên có cơ chế kiểm tra định kỳ và chủ động tái kết nối khi phát hiện lỗi. Đồng thời, các bản ghi trên Serial Monitor cũng rất cần thiết để theo dõi tình trạng hệ thống trong giai đoạn thử nghiệm và sửa lỗi.

Việc bổ sung cơ chế tái kết nối cho thấy đề tài không chỉ dừng lại ở mức “điều khiển được”, mà còn quan tâm đến độ tin cậy của hệ thống. Đây là yếu tố làm cho bài tiểu luận trở nên chín chu hơn về phương diện kỹ thuật, dù vẫn nằm trong phạm vi một mô hình mô phỏng học thuật.

5. KẾT QUẢ MÔ PHỎNG VÀ ĐÁNH GIÁ

5.1. Môi trường mô phỏng và cách thức kiểm thử

Toàn bộ mô hình được kiểm thử trong môi trường Wokwi, nơi ESP32, cảm biến và LED mô phỏng có thể hoạt động cùng nhau theo sơ đồ mạch đã cấu hình. Mã nguồn được biên dịch và nạp vào mô phỏng thông qua PlatformIO, giúp quy trình thử nghiệm phản ánh tương đối sát cách triển khai thực tế.

Quá trình kiểm thử được tổ chức theo từng nhóm tình huống: điều khiển từ Blynk, điều khiển qua Telegram, tác động từ nút nhấn vật lý và phản ứng tự động theo cảm biến. Cách kiểm thử phân nhóm giúp việc đánh giá trở nên rõ ràng, tránh bỏ sót các trường hợp dễ phát sinh lỗi đồng bộ.

Bảng 3. Kịch bản kiểm thử và kết quả quan sát trong mô phỏng

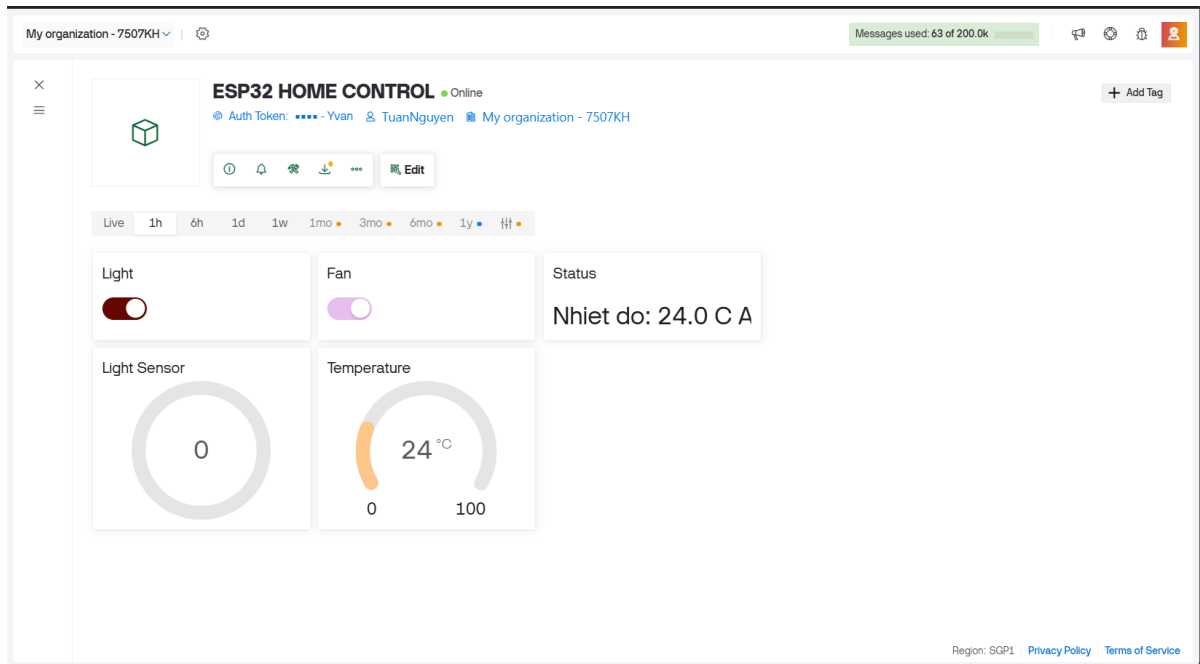
ST T	Kịch bản kiểm thử	Kết quả mong đợi	Đánh giá
1	Bật/tắt đèn từ Blynk	LED đèn đổi trạng thái và giao diện cập nhật đúng	Đạt

2	Bật/tắt quạt từ Telegram	ESP32 xử lý lệnh, gửi phản hồi và cập nhật Blynk	Đạt
3	Nhấn nút thủ công trên mô phỏng	Thiết bị thay đổi trạng thái, các nền tảng đồng bộ	Đạt
4	Thay đổi giá trị DHT22/LDR	Hệ thống kích hoạt hành vi tự động theo ngưỡng	Đạt
5	Giả lập mất kết nối tạm thời	Chương trình phát hiện lỗi và thực hiện tái kết nối	Đạt ở mức cơ bản

Sau khi hoàn thành việc cấu hình phần mềm, kết nối mạng Wi-Fi và tích hợp các dịch vụ Blynk, Telegram với ESP32, hệ thống đã được kiểm tra trong môi trường mô phỏng. Kết quả cho thấy mô hình có thể hoạt động đúng theo yêu cầu đặt ra, bao gồm chức năng điều khiển thiết bị từ xa, hiển thị dữ liệu cảm biến và cập nhật trạng thái thiết bị theo thời gian thực.

5.2. Giao diện điều khiển trên Blynk sau khi kết nối thành công

Sau khi ESP32 kết nối thành công với nền tảng Blynk, giao diện điều khiển trên web hiển thị đầy đủ các thành phần chính của hệ thống như công tắc điều khiển đèn, công tắc điều khiển quạt, vùng hiển thị trạng thái, cảm biến ánh sáng và nhiệt độ. Điều này cho thấy việc truyền dữ liệu giữa ESP32 và Blynk đã được thực hiện đúng, đồng thời xác nhận rằng người dùng có thể thao tác điều khiển thiết bị từ xa thông qua giao diện trực quan.

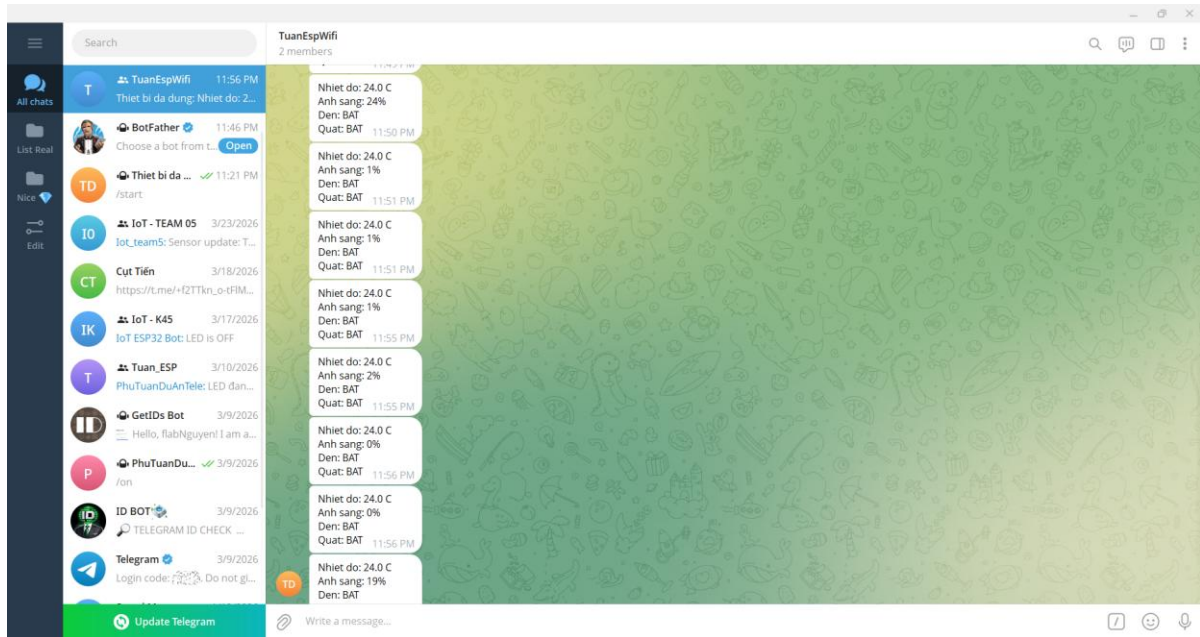


Từ giao diện này có thể thấy:

- Trạng thái kết nối của thiết bị ở mức **Online**.
- Các widget điều khiển như **Light**, **Fan** đã sẵn sàng để gửi lệnh đến ESP32.
- Các thông số như **Temperature**, **Light Sensor** và **Status** được cập nhật trên dashboard, hỗ trợ người dùng theo dõi tình trạng hoạt động của hệ thống.

5.3. Kết quả phản hồi và điều khiển qua Telegram

Bên cạnh giao diện Blynk, hệ thống còn hỗ trợ điều khiển từ xa thông qua Telegram Bot. Khi người dùng gửi các lệnh như bật đèn, tắt đèn, bật quạt, tắt quạt hoặc yêu cầu kiểm tra trạng thái, ESP32 sẽ tiếp nhận lệnh, thực hiện xử lý và gửi phản hồi trở lại qua Telegram. Điều này giúp tăng tính linh hoạt của hệ thống, đặc biệt trong trường hợp người dùng không truy cập trực tiếp vào giao diện Blynk.



Thông qua quá trình thử nghiệm, có thể nhận thấy rằng việc kết hợp Blynk và Telegram giúp hệ thống mở rộng khả năng tương tác với người dùng. Blynk phù hợp cho việc giám sát trực quan, trong khi Telegram thuận tiện cho việc điều khiển nhanh bằng tin nhắn.

Bảng 3. Kịch bản kiểm thử và kết quả quan sát trong mô phỏng

ST T	Kịch bản kiểm thử	Kết quả mong đợi	Đánh giá
1	Bật/tắt đèn từ Blynk	LED đèn đổi trạng thái và giao diện cập nhật đúng	Đạt
2	Bật/tắt quạt từ Telegram	ESP32 xử lý lệnh, gửi phản hồi và cập nhật Blynk	Đạt
3	Nhấn nút thử công trên mô phỏng	Thiết bị thay đổi trạng thái, các nền tảng đồng bộ	Đạt
4	Thay đổi giá trị DHT22/LDR	Hệ thống kích hoạt hành vi tự động theo ngưỡng	Đạt
5	Giả lập mất kết nối tạm thời	Chương trình phát hiện lỗi và thực hiện tái kết nối	Đạt ở mức cơ bản

5.4. Kết quả đạt được

Kết quả mô phỏng cho thấy hệ thống có thể điều khiển đèn và quạt theo đúng yêu cầu đặt ra. Giao diện Blynk hiển thị được các thành phần điều khiển cơ bản và phản ánh trạng thái thiết bị tương đối ổn định. Trong khi đó, Telegram cho phép nhận lệnh và trả về tin nhắn xác nhận, góp phần đa dạng hóa phương thức thao tác với hệ thống.

Dữ liệu từ DHT22 và LDR được đọc thành công và có thể dùng làm điều kiện kích hoạt tự động. Điều này chứng minh mô hình không chỉ dừng ở chức năng điều khiển thủ công, mà còn có khả năng phản ứng theo dữ liệu môi trường – một đặc trưng quan trọng của hệ thống IoT.

5.5. Ưu điểm của mô hình

- Chi phí triển khai thấp, phù hợp với môi trường học tập và nghiên cứu cơ bản.
- Giao diện điều khiển trực quan, dễ thao tác trên cả web, điện thoại và ứng dụng nhắn tin.
- Dễ mở rộng thêm cảm biến, relay hoặc các thiết bị gia dụng khác khi cần nâng cấp hệ thống.
- Có thể kết hợp điều khiển thủ công và điều khiển tự động mà vẫn giữ được tư duy đồng bộ trạng thái.

5.6. Hạn chế còn tồn tại

- Hệ thống hiện chủ yếu dừng ở mức mô phỏng, chưa đánh giá đầy đủ yếu tố nhiễu, độ bền và an toàn điện của phần cứng thật.
- Kết nối mạng vẫn là điều kiện bắt buộc; khi Wi-Fi mất ổn định, tốc độ phản hồi của hệ thống sẽ bị ảnh hưởng.
- Chưa triển khai cơ chế bảo mật sâu như mã hóa riêng, phân quyền người dùng hoặc lưu nhật ký truy cập chi tiết.
- Phần điều khiển tự động mới sử dụng ngưỡng đơn giản, chưa có cơ chế học thói quen hoặc tối ưu dựa trên dữ liệu lịch sử.

5.7. Hướng phát triển

Trong các phiên bản tiếp theo, hệ thống có thể được mở rộng theo nhiều hướng. Trước hết là thay LED mô phỏng bằng relay để điều khiển tải thực như bóng đèn, quạt

bàn hoặc ổ cắm thông minh. Kể đến, có thể bổ sung cảm biến chuyển động, cảm biến khí gas hoặc đồng hồ đo điện năng để nâng cao tính ứng dụng. Về mặt truyền thông, giao thức MQTT là lựa chọn đáng cân nhắc nếu hệ thống cần xử lý số lượng thiết bị lớn hơn hoặc yêu cầu phản hồi có tính thời gian thực cao hơn.

Ngoài phân cứng và giao thức, một hướng phát triển quan trọng khác là bảo mật. Hệ thống nên được tách khóa cấu hình ra khỏi mã nguồn, bổ sung cơ chế xác thực người dùng và cân nhắc mã hóa kết nối ở mức cao hơn khi triển khai trong môi trường thực. Đây là những bước cần thiết để chuyển một mô hình học thuật thành một sản phẩm có khả năng ứng dụng trong đời sống.

KẾT LUẬN

Đề tài đã xây dựng được một mô hình điều khiển thiết bị gia dụng qua Wi-Fi sử dụng ESP32 theo đúng định hướng Smart Home cơ bản. Hệ thống cho phép bật hoặc tắt đèn, quạt từ xa, theo dõi các thông số môi trường cơ bản và đồng bộ trạng thái giữa nhiều kênh điều khiển khác nhau. Việc kết hợp Blynk với Telegram giúp mô hình vừa trực quan, vừa linh hoạt trong quá trình vận hành.

Thông qua quá trình nghiên cứu và mô phỏng, đề tài cho thấy ESP32 là nền tảng phù hợp cho các ứng dụng IoT ở quy mô nhỏ nhờ tích hợp sẵn kết nối không dây, hỗ trợ phần cứng phong phú và chi phí triển khai hợp lý. Bên cạnh đó, việc sử dụng PlatformIO và Wokwi cũng góp phần làm rõ quy trình phát triển một hệ thống nhúng hiện đại từ giai đoạn viết mã đến giai đoạn kiểm thử.

Dù còn những hạn chế nhất định do phạm vi mới dừng ở mô phỏng, mô hình vẫn tạo được nền tảng đủ tốt để tiếp tục mở rộng sang triển khai phần cứng thật, bổ sung relay điều khiển tải thực, nâng cao bảo mật hoặc chuyển sang giao thức truyền thông chuyên biệt hơn. Vì vậy, đề tài không chỉ đáp ứng yêu cầu học tập mà còn mở ra nhiều hướng nghiên cứu và ứng dụng thực tiễn cho các dự án IoT trong tương lai.

TÀI LIỆU THAM KHẢO

[1] Nguyễn Văn Hiếu (2021), Hướng dẫn lập trình ESP32 với Arduino IDE, NXB Khoa học Kỹ thuật.

Truy cập ngày 10/4/2026.

[2] Espressif Systems, ESP32 Series Datasheet và tài liệu kỹ thuật liên quan đến phát triển ứng dụng nhúng.

Truy cập ngày 10/4/2026.

[3] Blynk Documentation, tài liệu tham khảo cho cấu hình dashboard, virtual pins và cơ chế giao tiếp với thiết bị.

Truy cập ngày 10/4/2026.

[4] Telegram Bot API Documentation, tài liệu tham khảo cho việc nhận lệnh và gửi phản hồi qua bot.

Truy cập ngày 10/4/2026.

[5] Wokwi Documentation, tài liệu mô phỏng phần cứng và cấu hình môi trường thử nghiệm cho ESP32.

Truy cập ngày 10/4/2026.

[6] Tài liệu học phần Lập trình IoT và tài liệu thực hành về cảm biến, giao tiếp Wi-Fi, mô phỏng Wokwi.

Truy cập ngày 10/4/2026.